Solution: Sorting a Stack

Problem Statement

Given a stack, sort it using only stack operations (push and pop).

You can use an additional temporary stack, but you may not copy the elements into any other data structure (such as an array). The values in the stack are to be sorted in descending order, with the largest elements on top.

Examples

```
1. Input: [34, 3, 31, 98, 92, 23]
   Output: [3, 23, 31, 34, 92, 98]

2. Input: [4, 3, 2, 10, 12, 1, 5, 6]
   Output: [1, 2, 3, 4, 5, 6, 10, 12]

3. Input: [20, 10, -5, -1]
   Output: [-5, -1, 10, 20]
```

Solution

This problem can be solved by using a temporary stack as auxiliary storage. The algorithm takes an input stack and sorts it using a temporary stack `tmpStack`. The sorting process is done by continuously popping elements from the input stack and pushing them onto the `tmpStack` in sorted order, rearranging elements as necessary between the stacks until the original stack is empty.

This algorithm leverages the LIFO (last in, first out) nature of a stack. It removes elements from the original stack one by one and uses a second stack to keep the elements sorted. If the top element of the sorted stack is larger than the current element, it moves the larger elements back to the original stack until it finds the correct spot for the current element, at which point it pushes the current element onto the sorted stack. Because of this, the smaller elements end up at the bottom of the sorted stack and the largest element on the top, resulting in a stack sorted in descending order from top to bottom.

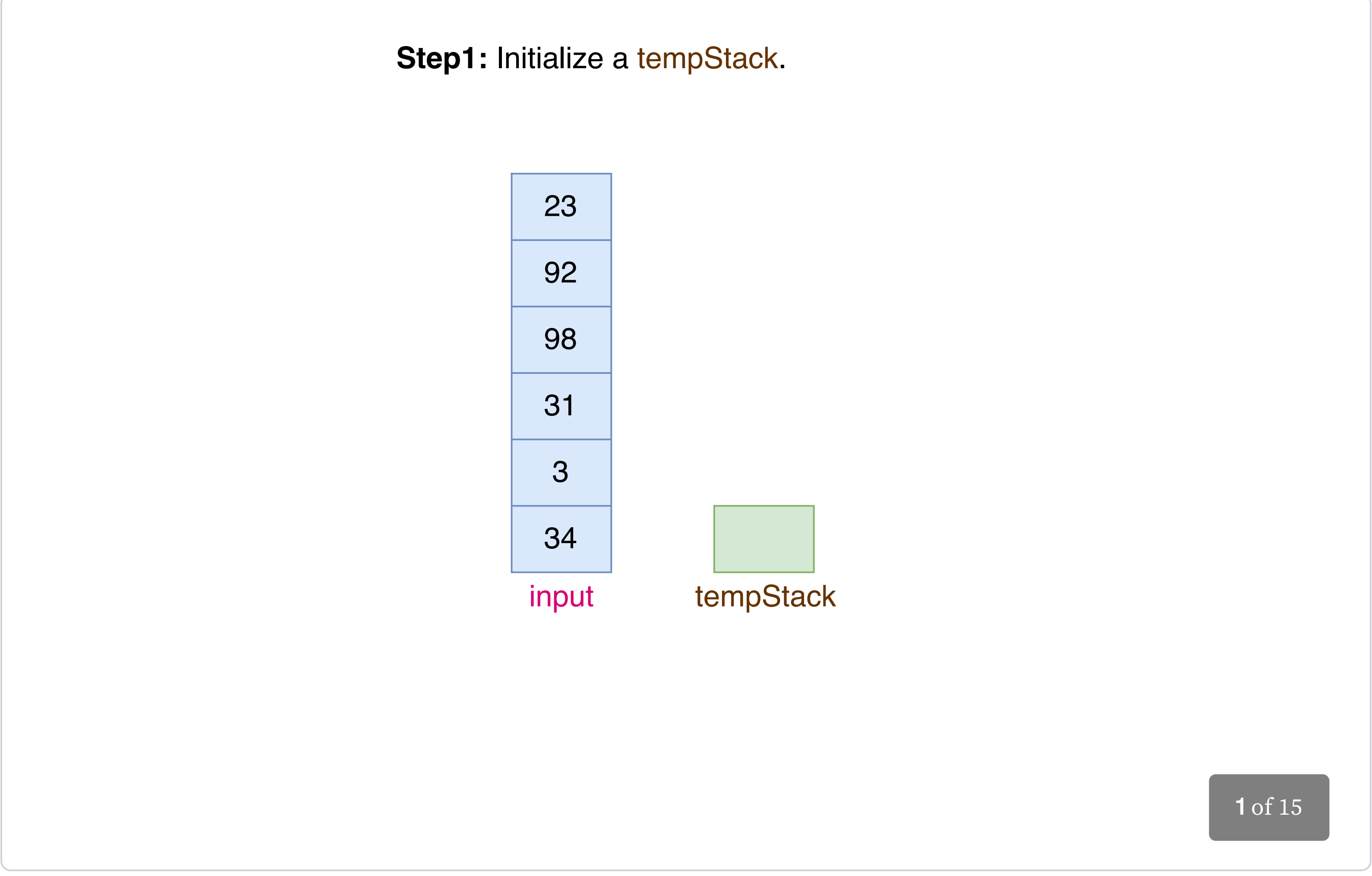
Detailed Step-by-Step Walkthrough

1. The `sort` method receives an input stack.
2. It initializes an empty temporary stack `tmpStack`.
3. The algorithm enters a loop that continues until the input stack is empty.
4. In each iteration, it pops the top element (`tmp`) from the input stack.
5. Then it enters another loop, which continues as long as `tmpStack` is not empty and the top of `tmpStack` is larger than `tmp`. In each iteration of this inner loop, it pops the top element from `tmpStack` and pushes it back onto the input stack.
6. After the inner loop ends, it pushes `tmp` onto `tmpStack`. The inner loop ensures that `tmpStack` is always sorted in descending order, with smaller elements at the bottom and larger elements at the top, and `tmp` is placed into its correct position in this order.
7. Once the outer loop ends and the input stack is empty, `tmpStack` contains all the elements originally in the input stack but sorted in descending order.
8. It then returns `tmpStack`.

Algorithm Walkthrough

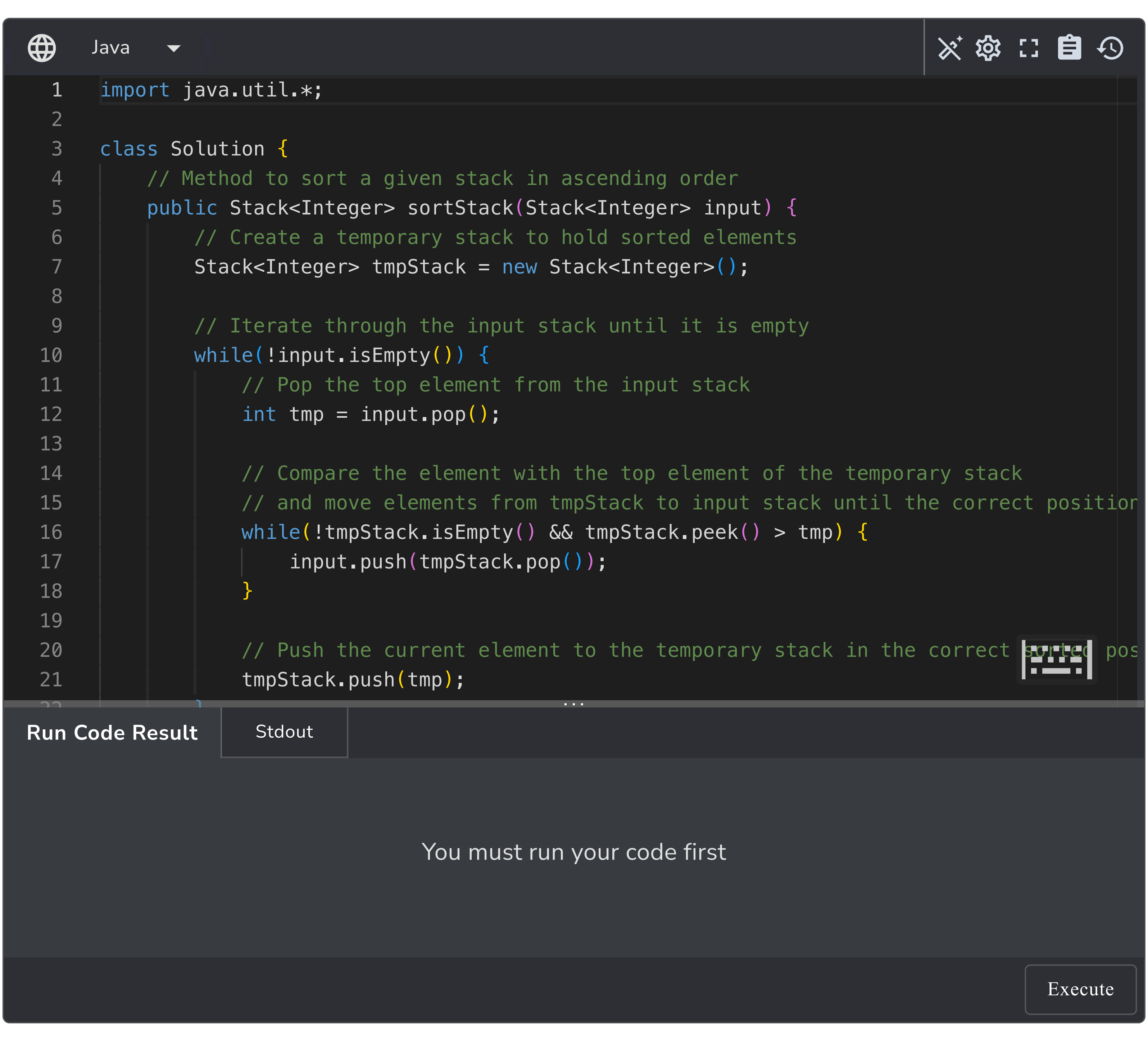
1. **Initial Setup**:
   - **Input Stack (top to bottom)**: `[34, 3, 31, 98, 92, 23]`
   - **Temporary Stack** (`tmpStack`): Empty
2. **Process Element**: **23**
   - Pop 23 from the input stack.
   - `tmpStack` is empty, so push 23 onto `tmpStack`.
   - **Input Stack**: `[34, 3, 31, 98, 92]`, **tmpStack**: `[23]`
3. **Process Element**: **92**
   - Pop 92 from the input stack.
   - Since 23 < 92, push 92 onto `tmpStack`.
   - **Input Stack**: `[34, 3, 31, 98]`, **tmpStack**: `[23, 92]`
4. **Process Element**: **98**
   - Pop 98 from the input stack.
   - Since 92 < 98, push 98 onto `tmpStack`.
   - **Input Stack**: `[34, 3, 31]`, **tmpStack**: `[23, 92, 98]`
5. **Process Element**: **31**
   - Pop 31 from the input stack.
   - Move elements from `tmpStack` to input stack until the correct position for 31 is found.
   - Pop 98, then 92 from `tmpStack` and push them onto the input stack.
   - Push 31 onto `tmpStack`.
   - **Input Stack**: `[34, 3, 98, 92]`, **tmpStack**: `[23, 31]`
   - In next 2 iterations, pop 98, and 92 from `input` stack and push back to the `tmpStack`. Updated stack will be: **Input Stack**: `[34, 3]`, **tmpStack**: `[23, 31, 92, 98]`.
6. **Process Element**: **3**
   - Pop 3 from the input stack.
   - Move elements from `tmpStack` to input stack until the correct position for 3 is found.
   - Pop 98, 92, 31, then 23 from `tmpStack` and push them onto the input stack.
   - Push 3 onto `tmpStack`.
   - **Input Stack**: `[34, 98, 92, 31, 23]`, **tmpStack**: `[3]`
     - In next 4 iterations, pop 98, 92, 31, and 23 from the `input` stack and push back to the `tmpStack`. Updated stack will be: **Input Stack**: `[34]`, **tmpStack**: `[3, 23, 31, 92, 98]` .
7. **Process Element**: **34**
   - Pop 34 from the input stack.
   - Move elements from `tmpStack` to input stack until the correct position for 34 is found.
   - Pop 98, then 92 from `tmpStack` and push them onto the input stack.
   - Push 34 onto `tmpStack`.
   - **Input Stack**: `[98, 92]`, **tmpStack**: `[3, 23, 31, 34]`.
   - In next 2 iterations, pop 98, and 92 from the `input` stack and push to the `tmpStack`.
8. **Final Result**:
   - **Input Stack**: Empty
   - **tmpStack (Sorted, top to bottom)**: `[3, 23, 31, 34, 92, 98]`

Here is the visual representation of the algorithm:



Code

Here is the code for this algorithm:



```java
import java.util.*;

class Solution {
    // Method to sort a given stack in ascending order
    public Stack<Integer> sortStack(Stack<Integer> input) {
        // Create a temporary stack to hold sorted elements
        Stack<Integer> tmpStack = new Stack<Integer>();

        // Iterate through the input stack until it is empty
        while(!input.isEmpty()) {
            // Pop the top element from the input stack
            int tmp = input.pop();

            // Compare the element with the top element of the temporary stack
            // and move elements from tmpStack to input stack until the correct positio
            while(!tmpStack.isEmpty() && tmpStack.peek() > tmp) {
                input.push(tmpStack.pop());
            }

            // Push the current element to the temporary stack in the correct sorted pos
            tmpStack.push(tmp);
```

Time and Space Complexity

Time Complexity

The time complexity of the sort stack algorithm is $O(n^2)$, where n is the number of elements in the stack. This is because, in the worst case, for every element that we pop from the input stack, we might have to pop all the elements in the temporary stack (and push them back to the original stack) to find the correct place to insert the element. Since we have to do this for all n elements, the time complexity is $n * n = n^2$.

Space Complexity

The space complexity of the algorithm is $O(n)$. This is because we use an additional temporary stack to sort the elements. In the worst-case scenario, this temporary stack could store all the elements of the original stack. Thus, the space complexity is directly proportional to the number of elements in the input stack, making it linear, $O(n)$.

Please note that the space complexity does not count the input size itself (the input stack in this case), it only counts the extra space we use related to the size of the input. If we were to count the input size as well, the total space used would be $O(2n)$, but since we drop the constant factor when expressing space complexity, so it remains $O(n)$.

← Previous
Problem 5: Sorting A Stack
Next →
Problem 6: Simplify Path
☐ Mark as Completed